РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

RPM Builder 1.4.1

От кода вашей программы до готового RPM-пакета — без ручного редактирования SPEC-файлов. Настроено для РЕД ОС и других RPM-дистрибутивов.

- **Установка за 2 минуты** — портативная версия запускается без установки: распакуйте архив и выполните **./rpm-builder.sh**.
- **Любой язык программирования** — скрипт, Python, C/C++, Go: главное — какие файлы и куда устанавливаются.
- **Сборка в один клик** — SPEC-файл генерируется автоматически, rpmbuild запускается с живым журналом.
- **Исправлено в 1.4.1** — сборка работает на реальных системах: рабочий каталог больше не содержит пробелов (это ломало rpmbuild), ошибки rpmbuild из stderr теперь видны в журнале, примеры собираются сразу после открытия.
- **Шаблоны проектов** — JSON-файлы (*.rpmb.json): сохраняйте и переносите настройки сборки.

ГЛАВА 1

Что такое RPM Builder

RPM Builder — визуальный конструктор RPM-пакетов. Вы написали программу и хотите распространять её по-взрослому: чтобы она ставилась командой `dnf install`, появлялась в `/usr/bin`, в меню приложений, корректно удалялась и отображалась в списке установленных пакетов системы. Для этого программа должна быть упакована в RPM — стандартный формат пакетов РЕД ОС, Fedora, RHEL, ALT Linux.

Классическая упаковка требует знания SPEC-файлов: служебных сценариев с директивами `%install`, `%files`, `%post` и десятком макросов. RPM Builder избавляет от этого этапа: вы заполняете карточку пакета, перетаскиваете файлы и нажимаете «Собрать». Приложение само разворачивает рабочее окружение rpmbuild (`SOURCES`, `SPECS`, `RPMS`…), генерирует SPEC, запускает сборку и показывает журнал в реальном времени, а готовый `.rpm` кладёт в выбранный каталог.

Приложение полностью автономно: портативная версия несёт собственные библиотеки Qt, работает на РЕД ОС 8/9, Fedora и других дистрибутивах с glibc 2.34+ и не требует ничего устанавливать. Единственное, что нужно для сборки — настоящий `rpmbuild` из пакета `rpm-build`, который есть в репозиториях РЕД ОС.

ГЛАВА 2

Установка на РЕД ОС

Есть три способа получить работающий RPM Builder. Выберите один — они не конфликтуют между собой. Первый способ самый быстрый и подходит большинству.

Способ А · Портативная версия — рекомендуется

Распакуйте архив и запустите. Ничего устанавливать не нужно, системные библиотеки не затрагиваются:

```
tar -xf rpm-builder-1.4.1-linux-x86_64.tar.xz
cd rpm-builder-portable
chmod +x rpm-builder.sh
./rpm-builder.sh
```

Требование: 64-битная система с glibc 2.34+ — это РЕД ОС 8/9, Fedora 35+, RHEL 9. Проверить свою версию: `ldd --version`. Если лаунчер сообщит о нехватке библиотек или у вас РЕД ОС 7.x — используйте способ Б.

Способ Б · Сборка из исходников — работает на любой РЕД ОС

Исходный код в каталоге `src/` собирается стандартными средствами дистрибутива — компилятор и Qt 6 берутся из репозиториев РЕД ОС:

```
# инструменты разработки + Qt6 (РЕД ОС / Fedora)
sudo dnf install gcc-c++ cmake ninja-build \
                 qt6-qtbase-devel qt6-qtdeclarative-devel

cd src
./build.sh              # готовый бинарник: build/rpm-builder
./build/rpm-builder
```

Скрипт `build.sh` сам находит Qt и собирает приложение. Если Qt 6 нет в вашем репозитории — установите Qt через `aqtinstall` (см. README.md в каталоге src) и укажите путь: `QT_DIR=/путь/к/Qt ./build.sh`.

Способ В · RPM-пакет RPM Builder'а — «как положено»

Приложение умеет собирать само себя. В корне архива лежит готовый `rpm-builder.spec`:

```
# один раз: окружение сборки
sudo dnf install rpm-build rpmdevtools && rpmdev-setuptree

# архив исходников в SOURCES (имя должно совпадать со spec)
tar -czf ~/rpmbuild/SOURCES/rpm-builder-1.4.1.tar.gz \
    --exclude=build --exclude=.git rpm-builder-1.4.1

rpmbuild -ba rpm-builder.spec
sudo dnf install ~/rpmbuild/RPMS/x86_64/rpm-builder-1.4.1-1.*.rpm
```

Каталог `rpm-builder-1.4.1` — это переименованный `src/` (имя должно совпадать с Source0 в spec). После установки RPM Builder появляется в меню приложений (категория «Разработка») и запускается командой `rpm-builder`. Удаление — `sudo dnf remove rpm-builder`.

Установщик в меню. Для способов А и Б добавьте ярлык в меню приложений командой `sudo ./install.sh` из каталога `src` (удаление — `sudo ./uninstall.sh`). Портативную версию можно так и оставить запускать через `./rpm-builder.sh` — это нормально.

Что установить для самой сборки пакетов

Единственная внешняя зависимость приложения — системный rpmbuild. Поставьте его сразу, а для упаковки C/C++-программ добавьте компилятор:

```
sudo dnf install rpm-build rpmdevtools
# для C/C++-программ дополнительно:
sudo dnf install gcc make
```

ГЛАВА 3

Интерфейс: семь разделов

Слева — навигация и карточка текущего проекта, справа — содержимое раздела. Тема переключается кнопкой в заголовке окна (тёмная и светлая), выбор запоминается. Внизу слева — счётчик сборок и результат последней из них.

РАЗДЕЛ	ЧТО ЗДЕСЬ ДЕЛАЕТСЯ
Обзор	Статистика, кнопка «Новый проект», недавние проекты — до 12 записей, загрузка в один клик.
Метаданные	Карточка пакета: имя, версия, релиз, лицензия, архитектура, сводка, описание. Валидатор сразу подсвечивает ошибки и объясняет, что не так.
Файлы пакета	Перетащите файлы и папки из проводника. Для каждого файла — каталог установки (подставляется автоматически, можно править).
Зависимости	Три списка чипами: Requires, BuildRequires, Provides.
Скрипты	Необязательные сценарии %pre, %post, %preun, %postun с кнопкой «Вставить пример».
SPEC-файл	Живое превью сгенерированного SPEC. Можно скопировать или сохранить как `.spec`.
Сборка	Кнопка «Собрать пакет», живой журнал rpmbuild, таймер, отмена, карточки готовых .rpm с кнопкой «Открыть папку».

> **Блок «ШАБЛОН ПРОЕКТА»** в панели слева (новое в 1.4.0) — кнопки «Сохранить в файл...» и «Открыть из файла...» для JSON-шаблонов. Подробно — в главе 7.

ГЛАВА 4

Главный сценарий: из кода — в RPM

Это ответ на вопрос «я написал программу — как её собрать в RPM?». Схема всегда одна и не зависит от языка: программа — это набор файлов; RPM-пакет — опись «какие файлы куда положить при установке». Подготовили файлы → описали их в RPM Builder → нажали «Собрать» → установили готовый пакет.

1 Подготовьте файлы программы

Соберите всё, что должно попасть в пакет. Если программа на скриптовом языке (shell, Python, Perl) — просто файл скрипта с правами на исполнение. Если на C/C++, Go, Rust — сначала скомпилируйте исполняемый файл: RPM-пакет содержит готовые бинарники, а не исходники.

```
# пример: программа на C
gcc -O2 -o myapp myapp.c
# пример: скрипт на Python
chmod +x myapp.py
```

Дополнительно можно приложить ярлык `.desktop` (для меню) и иконку `.svg/.png` — RPM Builder сам предложит для них правильные каталоги.

2 Заполните метаданные

Раздел «Метаданные». Обязательные поля подсвечиваются, пока не заполнены. Имя — строчными латинскими буквами, цифрами и дефисом; версия — начинается с цифры; релиз оставьте `1%{?dist}` (стандарт RPM-дистрибутивов).

ПОЛЕ	СМЫСЛ	ПРИМЕР
Имя пакета	Идентификатор для dnf/rpm	`myapp`
Версия	Версия вашей программы	`1.4.1`
Релиз	Номер пересборки этой версии	`1%{?dist}`
Лицензия	SPDX-код лицензии	`MIT`, `GPL-3.0-only`
Сводка	Одна строка, до 80 символов	`Утилита для отчётов`
Архитектура	x86_64 для бинарников; noarch для скриптов и данных	`x86_64`

3 Добавьте файлы и каталоги установки

Раздел «Файлы пакета»: перетащите файлы в пунктирную зону. Для каждого файла выберите, куда он должен попасть при установке. Каталог подставляется автоматически по типу файла, список подсказок содержит типовые макросы RPM. Файловая система Linux жёстко регулирует, что где должно лежать:

ФАЙЛ	КАТАЛОГ УСТАНОВКИ
Исполняемая программа, скрипт	`%{_bindir}` → /usr/bin — доступна по имени из терминала
Библиотеки (.so)	`%{_libdir}` → /usr/lib64
Ярлык в меню (.desktop)	`/usr/share/applications`
Иконка (.svg / .png)	`/usr/share/icons/hicolor/…`
Конфигурация (.conf)	`/etc/имя-пакета`
Документация (README, LICENSE)	`/usr/share/doc/%{name}`
Прочие ресурсы программы	`/usr/share/%{name}`

Указывается каталог назначения; имя файла при установке сохраняется. Хотите команду без расширения — назовите файл `myapp`, а не `myapp.py`.

4 Укажите зависимости

Раздел «Зависимости». **Requires** — что должно быть в системе для работы вашей программы (интерпретатор python3, библиотека, утилита). **BuildRequires** — что нужно на этапе сборки (для упаковки готовых файлов обычно пусто). **Provides** — что ваш пакет объявляет другим. Зависимости можно указывать с версией: `python3 ⩾ 3.9`.

Скрипт на Python → Requires: `python3`. Скрипт на shell → `bash`. Программа на C, собранная статически, может обойтись без зависимостей; у динамического бинарника их подскажет `ldd myapp`.

5 Скрипты и сборка — при необходимости

Раздел «Скрипты и сборка» необязателен. Вкладки `%pre`, `%post`, `%preun` и `%postun` — скриплеты установки/удаления (сообщение пользователю, обновление кэшей); для простых программ оставьте их пустыми. Вкладки `%build`, `%install` и `%files` — для сборки пакетов из исходников (Rust, Go, C, make): компиляция выполняется во время сборки RPM. Подробно — в главе 6. Кнопка «Вставить пример» заполнит любую вкладку готовым текстом.

6 Проверьте SPEC-превью

Раздел «SPEC-файл» показывает ровно то, что получит rpmbuild: заголовок с метаданными, списки SourceN и Requires, блок %install (копирование ваших файлов), ваши скрипты и %files. Любое изменение формы мгновенно обновляет превью — это и учебник, и предохранитель. Здесь же SPEC можно сохранить отдельным файлом, например для CI.

7 Соберите пакет

Раздел «Сборка»: выберите каталог результатов (по умолчанию `~/rpmbuilder-output`) и нажмите «Собрать пакет». RPM Builder проверит форму, развернёт окружение, скопирует файлы, сгенерирует SPEC и запустит rpmbuild — весь журнал виден построчно, этапы пронумерованы [1/6]…[6/6]. При ошибке её строки подсвечиваются, сборка останавливается: исправьте то, на что указал журнал, и запустите снова.

Результат: `~/rpmbuilder-output/имя-версия-релиз.архитектура.rpm` — карточка артефакта с кнопкой «Открыть папку» появится прямо на странице.

8 Установите и проверьте

```
# установка (dnf сам подтянет зависимости)
sudo dnf install ~/rpmbuilder-output/myapp-1.4.1-1.*.x86_64.rpm

# проверка
myapp                       # запустите программу по имени
rpm -qi myapp               # карточка пакета
rpm -ql myapp               # список файлов пакета в системе

# удаление
sudo dnf remove myapp
```

Выпустили новую версию? Поменяйте «Версию» в метаданных (релиз верните в `1%{?dist}`) и повторите сборку. Пересобрали ту же версию — увеличьте релиз: `2%{?dist}`. Обновление установленного пакета: `sudo dnf upgrade ./файл.rpm`.

ГЛАВА 5

Быстрый старт за пять минут: пример hello

В архиве есть готовый учебный проект — `examples/hello/`. Это скрипт `hello.sh`, шаблон проекта `hello.rpmb.json` и эталонный SPEC. Пройдите его целиком — после этого собственную программу вы соберёте по тем же шагам. С версии 1.4.1 копировать ничего не нужно: при открытии шаблона файлы подхватываются из его папки автоматически.

1. Запустите RPM Builder и в панели слева нажмите **«Открыть из файла...»** → выберите `hello.rpmb.json` прямо из папки `examples/hello/`. Форма заполнится сама: имя `hello`, версия 1.0, архитектура noarch (скрипту без разницы, где исполняться), зависимость `bash`, скрипт %post с сообщением и файл `hello.sh` (путь подставится автоматически — тост это подтвердит) → `%{_bindir}`.

2. Откройте раздел «Сборка» и нажмите **«Собрать пакет»**. В журнале пройдут этапы [1/6]...[6/6], появится карточка `hello-1.0-1.*.noarch.rpm`.

```
# 3. установите и проверьте
sudo dnf install ~/rpmbuilder-output/hello-1.0-1.*.noarch.rpm
hello.sh
# Привет из RPM-пакета hello! Версия 1.0
```

Удаление учебного пакета: `sudo dnf remove hello`. Файлы примера подробно описаны в `examples/hello/ПРОЧТИ-МЕНЯ.txt`.

ГЛАВА 6

Пример: программа на C (или другом компилируемом языке)

Для компилируемых языков есть два пути. **Простой**: скомпилируйте программу сами (`gcc -O2 -o myapp myapp.c`) и упакуйте готовый бинарник — как в главе 4. **Нативный (новое в 1.4.0)**: положите в пакет исходники, и компиляция выполнится сама во время сборки RPM. Разбираем нативный путь — он же работает для Rust (`cargo build`), Go (`go build`) и любых make/cmake-проектов.

```
// myapp.c — минимальная программа
#include <stdio.h>
int main(void) {
    printf("Привет из пакета myapp!\n");
    return 0;
}
```

В RPM Builder: имя `myapp` , версия `1.0` , архитектура `x86_64` . В «Файлы пакета» перетащите исходник `myapp.c` — он попадёт в каталог источников `%{_sourcedir}` ; каталог установки для исходников не нужен. В «Зависимости» добавьте в **BuildRequires** инструмент сборки: `gcc` . Дальше — страница «Скрипты и сборка», три новые вкладки:

```
# %build — компиляция во время сборки RPM
cd %{_sourcedir}
gcc -O2 %{_sourcedir}/myapp.c -o myapp

# %install — что и куда положить в пакет
install -Dm755 %{_sourcedir}/myapp %{buildroot}%{_bindir}/myapp

# %files — список файлов пакета
%{_bindir}/myapp
```

Кнопка «Вставить пример» подставит заготовки, включая вариант для Rust/Go. SPEC пересоберётся сам: появится секция %build с вашими командами. После нажатия «Собрать пакет» в журнале видно, как rpmbuild компилирует программу — готовый бинарник никуда вручную копировать не нужно. Те же три вкладки сохраняются в JSON-шаблон и в историю проектов.

Rust и Go. Перетащите на «Файлы» **всю папку** проекта (она целиком окажется в %{_sourcedir}), в BuildRequires укажите `cargo` или `golang`, а во вкладке %build напишите: `cd %{_sourcedir}/myapp && cargo build --release`; в %install возьмите бинарник из `target/release/` (Rust) или корня проекта (Go). Готовый разбор с файлами — в архиве: `examples/hello-c/ПРОЧТИ-МЕНЯ.txt`. Простой путь «упаковка готового бинарника» никуда не делся: оставьте три вкладки пустыми — всё работает как в главе 4.

ГЛАВА 7

JSON-шаблоны проектов (*.rpmb.json)

Шаблон — это весь проект одним файлом: метаданные, список файлов с каталогами установки, зависимости и скрипты. Удобно, когда пакетов много и они однотипны: сделали один эталонный шаблон — и каждая следующая сборка занимает минуту. Шаблоном удобно делиться с коллегами или хранить в git-репозитории проекта.

Сохранить в файл… — текущий проект записывается в `имя-версия.rpmb.json` (по умолчанию — в «Документы»). **Открыть из файла…** — форма полностью восстанавливается из шаблона, проект попадает в черновик, а приложение переключается на «Метаданные». После загрузки проверьте раздел «Файлы пакета»: валидатор отметит файлы, которых нет на этом компьютере, — перетащите их заново, остальное останется как было.

Шаблоны переносимы между машинами: пути вида `~/…` раскрываются на той системе, где шаблон открывают. Формат — обычный JSON, его можно править в любом текстовом редакторе:

```
{
  "magic": "rpm-builder-project",
  "formatVersion": 1,
  "appVersion": "1.4.1",
  "project": {
    "name": "hello", "version": "1.0", "release": "1%{?dist}",
    "license": "MIT", "arch": "noarch",
    "files": [ { "src": "~/hello.sh", "dst": "%{_bindir}" } ],
    "requires": [ "bash" ],
    "scripts": { "post": "echo Готово!" }
  }
}
```

Поле `magic` обязательно — файлы без него приложение не примет. В архиве есть готовые примеры: `examples/hello/hello.rpmb.json` (простая упаковка) и `examples/hello-c/hello-c.rpmb.json` (сборка из исходников).

ГЛАВА 8

RPM Builder сам в RPM и GitHub Actions

Проект самодостаточен: в корне лежит `rpm-builder.spec` — SPEC для упаковки самого приложения в RPM (способ В из главы 2), а в каталоге `.github/workflows/` — файл `build.yml` для GitHub Actions. Если исходники размещены на GitHub, workflow автоматически: собирает приложение в контейнере Fedora, прогоняет оффлайн-самопроверку (`--selftest`, включая тест JSON-шаблонов), собирает self-RPM и выкладывает бинарник и RPM-пакеты в артефакты сборки. Запуск — на каждый push в main/master или вручную кнопкой **Run workflow**.

Даже без GitHub этот workflow полезен как справочник: в нём в точности записаны команды установки зависимостей и сборки для Fedora — те же команды работают и на РЕД ОС с минимальными правками имён пакетов.

ГЛАВА 9

Частые проблемы и решения

СИМПТОМ	РЕШЕНИЕ
«rpmbuild не найден» при сборке	`sudo dnf install rpm-build`. Проверить: `rpmbuild --version`.
Ошибка «version GLIBC_x.xx not found» при запуске portable	Система старше требуемой (нужно glibc 2.34+, то есть РЕД ОС 8+). Соберите из исходников — способ Б из главы 2.
Окно не открывается, «Could not load the Qt platform plugin»	Не хватает системных библиотек X11: `sudo dnf install libxcb xcb-util-cursor`. В сессии Wayland приложение работает через xcb-совместимость.
В шаблоне, принесённом с другого ПК, файлы «не найдены»	С версии 1.4.1 файлы автоматически подхватываются из папки самого шаблона (*.rpmb.json), если они лежат рядом с ним. Иначе перетащите файлы заново на странице «Файлы пакета» — метаданные и зависимости сохранятся.
«файл не найден» в журнале rpmbuild	Проверьте пути в «Файлах пакета» (валидатор перед сборкой тоже предупредит) и что файл существует прямо сейчас.
Скрипт %post падает при установке	Скрипты выполняются sh. Упростите до одной-двух команд, проверьте их вручную в

	терминале, убедитесь в отсутствии конфликтов имён.
dnf отказывается обновлять: «does not update installed package»	Версия/релиз не больше установленного. Поднимите релиз (`2%{?dist}`) или версию.
Пакет собрался, а ярлыка в меню нет	Добавьте в пакет `.desktop`-файл и `.svg/.png`-иконку (глава 4, шаг 3), затем `sudo update-desktop-database`.
Сборка падала «непонятно почему» (до 1.4.1)	Причина была в пробеле в рабочем каталоге (.../RPM Builder/workspace) — rpm его не переносит. В 1.4.1 каталог перенесён в `~/.local/share/rpm-builder/workspace` без пробелов, а все ошибки rpmbuild (stderr) теперь видны в журнале. Просто обновите приложение.

> **Ничего не сломается.** Установка/удаление RPM-пакета не трогает чужие файлы: пакет владеет только тем, что перечислено в %files, — и вы это видели в SPEC-превью. SELinux в РЕД ОС мешать не будет: установка идёт штатным dnf, который сам расставляет правильные контексты безопасности.

ГЛАВА 10

Шпаргалка команд

```
# --- установка и удаление пакетов ---
sudo dnf install ./пакет.rpm        # установить локальный rpm
sudo dnf upgrade ./пакет.rpm        # обновить до новой версии
sudo dnf remove имя                  # удалить пакет

# --- информация ---
rpm -qi имя                          # карточка: версия, лицензия, описание
rpm -ql имя                          # какие файлы поставил пакет
rpm -qf /usr/bin/hello               # какому пакету принадлежит файл

# --- сборка руками (что делает RPM Builder за вас) ---
rpmdev-setuptree                     # создать ~/rpmbuild/...
rpmbuild -ba my.spec                 # собрать пакет из SPEC
rpmbuild --rebuild src.rpm           # пересобрать из src.rpm

# --- RPM Builder ---
./rpm-builder.sh                     # запуск portable
./rpm-builder.sh --selftest          # проверка конвейера без окна
```

Подробный справочник: `man rpmbuild`, `man spec`. RPM Builder показывает готовый SPEC на странице «SPEC-файл» — это лучший способ изучить формат на живом примере.

ПРИЛОЖЕНИЕ

Состав архива

Каждый файл на своём месте. Если что-то не запустилось — главы 2 и 9 содержат обходные пути для любой ситуации.

`rpm-builder-portable/`	готовая портативная версия: **./rpm-builder.sh** — и работает (глава 2, способ A)
`src/`	исходный код v1.4.1: build.sh, README.md, .github/workflows для CI
`rpm-builder.spec`	сборка RPM Builder'а в RPM-пакет (глава 2, способ B)
`examples/hello/`	учебный проект: hello.sh, hello.rpmb.json, hello.spec, ПРОЧТИ-МЕНЯ.txt (глава 5)
`docs/screenshots/`	скриншоты интерфейса в двух темах
`РУКОВОДСТВО-RU.pdf`	этот документ — читайте с экрана или распечатайте
`SHA256SUMS.txt`	контрольные суммы для проверки целостности

Лицензия MIT. Шрифты Inter и JetBrains Mono — SIL OFL 1.1.
Собирайте пакеты с удовольствием.